

24CSI024-DATA STRUCTURES AND ALGORITHMS

UNIT 1-INTRODUCTION TO DATA STRUCTURES AND ALGORITHMS

SORTING

Pre Lab Task

Learn the following constructs:

1. Loops
2. Conditional Statements
3. Functions
4. File Handling

In Lab exercises

1. Implementation of Insertion sort using array.

CODE:

```
1  #include <stdio.h>
2  int main() {
3      int n;
4      printf("Enter the number of elements: ");
5      scanf("%d", &n);
6      int a[n];
7      printf("Enter %d elements:\n", n);
8      for(int i = 0; i < n; i++)
9          scanf("%d", &a[i]);
10
11     for(int i = 1; i < n; i++) {
12         int key = a[i];
13         int j = i - 1;
14         while(j >= 0 && a[j] > key) {
15             a[j + 1] = a[j];
16             j--;
17         }
18         a[j + 1] = key;
19     }
20
21     printf("\nSorted Array:\n");
22     for(int i = 0; i < n; i++)
23         printf("%d ", a[i]);
24
25     return 0;
26 }
```

OUTPUT:

main.c

Output

```
Enter the number of elements: 6
```

```
Enter 6 elements:
```

```
9
```

```
4
```

```
7
```

```
2
```

```
8
```

```
1
```

```
Sorted Array:
```

```
1 2 4 7 8 9
```

2. Given an integer array `nums` and an integer `k`, return *the k^{th} largest element in the array*. Note that it is the k^{th} largest element in the sorted order, not the k^{th} distinct element. Can you solve it without sorting?

Example 1:

Input: `nums = [3,2,1,5,6,4]`, `k = 2`

Output: 5

Example 2:

Input: `nums = [3,2,3,1,2,4,5,5,6]`, `k = 4`

Output: 4

CODE:

Edit**Preview**

```
1  #include <stdio.h>
2  #include <limits.h>
3  int main() {
4      int n, k;
5      printf("Enter the number of elements: ");
6      scanf("%d", &n);
7      int a[n];
8      printf("Enter %d elements:\n", n);
9      for(int i = 0; i < n; i++)
10         scanf("%d", &a[i]);
11     printf("Enter the value of k: ");
12     scanf("%d", &k);
13     for(int i = 0; i < k; i++) {
14         int max = INT_MIN;
15         int index = -1;
16         for(int j = 0; j < n; j++) {
17             if(a[j] > max) {
18                 max = a[j];
19                 index = j;
20             }
21         }
22         if(i == k - 1)
23             printf("\nThe %dth largest element is: %d", k, max);
24         a[index] = INT_MIN;
25     }
26     return 0;
27 }
```

OUTPUT:**main.c****Output**

Enter the number of elements: 6

Enter 6 elements:

3

2

1

5

6

4

Enter the value of k: 2

The 2th largest element is: 5

=== Code Execution Successful ===

3. Given a string `s`, sort it in decreasing order based on the frequency of the characters. The frequency of a character is the number of times it appears in the string. Return the sorted string. If there are multiple answers, return any of them.

Example 1:

Input: `s = "tree"`

Output: `"eert"`

Explanation: 'e' appears twice while 'r' and 't' both appear once.

So 'e' must appear before both 'r' and 't'. Therefore `"eetr"` is also a valid answer.

Example 2:

Input: `s = "cccaaa"`

Output: `"aaaccc"`

Explanation: Both 'c' and 'a' appear three times, so both `"cccaaa"` and `"aaaccc"` are valid answers.

Note that `"cacaca"` is incorrect, as the same characters must be together.

Example 3:

Input: s = "Aabb"

Output: "bbAa"

Explanation: "bbaA" is also a valid answer, but "Aabb" is incorrect.

Note that 'A' and 'a' are treated as two different characters.

Constraints:

$1 \leq s.length \leq 5 * 10^5$

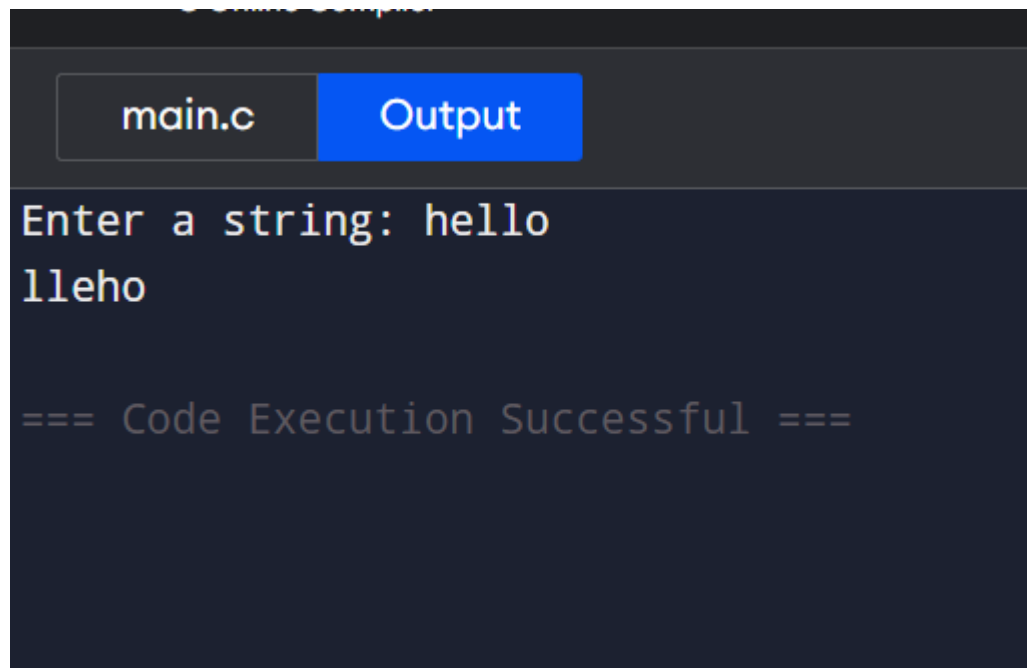
s consists of uppercase and lowercase English letters and digits.

CODE:

```

1  #include <stdio.h>
2  #include <string.h>
3  int main() {
4      char s[1000];
5      int freq[256] = {0};
6      printf("Enter a string: ");
7      scanf("%s", s);
8      for(int i = 0; s[i] != '\0'; i++)
9          freq[(unsigned char)s[i]]++;
10     while(1) {
11         int max = 0;
12         int ch = -1;
13         for(int i = 0; i < 256; i++) {
14             if(freq[i] > max) {
15                 max = freq[i];
16                 ch = i;
17             }
18         }
19         if(ch == -1)
20             break;
21         for(int i = 0; i < max; i++)
22             printf("%c", ch);
23         freq[ch] = 0;
24     }
25     return 0;
26 }
```

OUTPUT:



The screenshot shows a code editor window with two tabs: 'main.c' and 'Output'. The 'main.c' tab is active, displaying the following C code:

```
#include <stdio.h>
int main()
{
    char str[100];
    printf("Enter a string: ");
    gets(str);
    printf("%s", str);
    return 0;
}
```

 The 'Output' tab is also visible, showing the program's execution:

```
Enter a string: hello
lleho

=== Code Execution Successful ===
```

Post Lab exercises:

1. Merge Sort
2. Bubble sort
3. Selection Sort